

Лекция 8: Подпрограммы (Методы)

Введение

До сих пор весь наш код помещался внутри метода `main()`. Это работало для небольших примеров, но по мере роста программы такой подход приводит к проблемам:

- Код становится длинным и трудночитаемым
- Один и тот же фрагмент кода приходится копировать в разные места
- Трудно тестировать и отлаживать программу
- Сложно разделить работу между несколькими разработчиками

Подпрограммы (методы) — это способ организации кода, при котором программа разбивается на небольшие логические блоки, каждый из которых решает конкретную задачу. Методы позволяют:

- **Повторно использовать** код (написал один раз — вызывай много раз)
- **Упрощать** чтение и понимание программы (метод делает одно действие)
- **Упрощать** тестирование и отладку (можно проверить каждый метод отдельно)
- **Скрывать** сложность (пользователю метода не важно, как он работает внутри)

Сегодня мы изучим:

- Что такое методы и зачем они нужны
- Объявление и вызов методов
- Параметры и возвращаемые значения
- Перегрузка методов
- Рекурсия (метод вызывает сам себя)

1) Что такое методы

В Java метод — это блок кода, который имеет имя и может быть вызван из других мест программы. Методы всегда находятся внутри класса.

Простейший метод (без параметров и без возврата)

```
public class SimpleMethod {  
    public static void main(String[] args) {  
        // Вызов метода  
        printWelcome();  
        printWelcome();  
        printWelcome();  
    }  
}
```

```

    }

    // Объявление метода
    public static void printWelcome() {
        System.out.println("Добро пожаловать в программу!");
        System.out.println("-----");
    }
}

```

Разбор синтаксиса объявления метода:

Часть	Значение	Пример
public	Модификатор доступа (кто может вызвать)	открытый для всех
static	Статический метод (принадлежит классу, а не объекту)	можно вызвать без создания объекта
void	Возвращаемый тип (void — ничего не возвращает)	нет возвращаемого значения
printWelcome	Имя метода (глагол, с маленькой буквы, camelCase)	как назвали
()	Список параметров (пустые — нет параметров)	
{...}	Тело метода (код, который выполняется)	

Таблица 1. Элементы объявления метода

Почему static?

На данном этапе мы используем static, чтобы вызывать методы из main() без создания объекта класса. В следующих курсах (ООП) вы узнаете, что такое объекты, и сможете создавать нестатические методы.

2) Методы с параметрами

Параметры — это данные, которые метод получает для работы. Они указываются в круглых скобках после имени метода.

```

public class MethodWithParams {
    public static void main(String[] args) {
        printMessage("Привет, мир!");
        printMessage("Java — отличный язык");

        printSum(5, 3);
        printSum(10, 20);
    }

    // Метод с одним параметром типа String
    public static void printMessage(String text) {
        System.out.println("Сообщение: " + text);
    }

    // Метод с двумя параметрами типа int
    public static void printSum(int a, int b) {
        int sum = a + b;
        System.out.println(a + " + " + b + " = " + sum);
    }
}

```

Параметры — это локальные переменные внутри метода. Они существуют только пока выполняется метод, и недоступны снаружи.

Несколько параметров разных типов

```

public class MultipleParams {
    public static void main(String[] args) {
        printProduct("Квадрат", 5, 3.5);
        printProduct("Прямоугольник", 10, 2.0);
    }

    public static void printProduct(String name, int a, double b) {
        double result = a * b;
        System.out.println(name + ": " + a + " × " + b + " = " + result);
    }
}

```

3) Методы с возвращаемым значением

Метод может не только что-то делать, но и **возвращать** результат вычислений. Вместо void указывается тип возвращаемого значения, а внутри метода используется ключевое слово return.

```
public class ReturnMethod {
    public static void main(String[] args) {
        int x = 5, y = 3;

        int sum = add(x, y);
        System.out.println(x + " + " + y + " = " + sum);

        int maxValue = max(10, 20);
        System.out.println("Максимум: " + maxValue);

        boolean isEven = isEven(7);
        System.out.println("Число 7 чётное? " + isEven);
    }

    // Возвращает int
    public static int add(int a, int b) {
        int result = a + b;
        return result; // возвращаем значение
    }

    // Возвращает int (более краткая запись)
    public static int max(int a, int b) {
        return (a > b) ? a : b;
    }

    // Возвращает boolean
    public static boolean isEven(int number) {
        return number % 2 == 0;
    }
}
```

Важные правила return:

1. Тип возвращаемого значения должен совпадать с объявленным (или быть совместимым):

```
public static double getValue() {
    return 5; // int автоматически расширяется до double — ОК
}
```

2. **Метод с возвращаемым значением должен обязательно вернуть значение во всех ветках:**

java

Копировать

Скачать

```
public static int abs(int x) {  
    if (x >= 0) {  
        return x;  
    } else {  
        return -x;  
    }  
}
```

3. **После return код не выполняется:**

```
public static void example() {  
    System.out.println("Это выполнится");  
    return;  
    System.out.println("Это НЕ выполнится"); // Ошибка компиляции!  
}
```

4. **Метод с void может использовать return для досрочного выхода:**

java

Копировать

Скачать

```
public static void printPositive(int x) {  
    if (x < 0) {  
        return; // выходим из метода, ничего не печатая  
    }  
    System.out.println(x);  
}
```

4) Перегрузка методов (Overloading)

Перегрузка — это возможность создавать несколько методов с **одинаковым именем**, но **разными параметрами** (количество, тип или порядок).

Компилятор сам определяет, какой метод вызвать, по переданным аргументам.

```
public class OverloadingDemo {
```

```

public static void main(String[] args) {
    // Вызываются разные методы в зависимости от аргументов
    print(5);        // вызывает print(int)
    print(5.5);      // вызывает print(double)
    print("Hello");  // вызывает print(String)
    print(3, 7);     // вызывает print(int, int)
}

public static void print(int x) {
    System.out.println("int: " + x);
}

public static void print(double x) {
    System.out.println("double: " + x);
}

public static void print(String x) {
    System.out.println("String: " + x);
}

public static void print(int a, int b) {
    System.out.println("int,int: " + a + ", " + b);
}
}

```

Зачем нужна перегрузка?

Перегрузка позволяет использовать одно и то же понятное имя для похожих действий:

```

public class MathOperations {
    // Разные типы параметров
    public static int square(int x) { return x * x; }
    public static double square(double x) { return x * x; }

    // Разное количество параметров
    public static int sum(int a, int b) { return a + b; }
    public static int sum(int a, int b, int c) { return a + b + c; }
    public static int sum(int a, int b, int c, int d) { return a + b + c + d; }
}

```

Важное правило: Перегрузка НЕ работает только по возвращаемому типу. Нельзя создать два метода с одинаковыми параметрами, но разными возвращаемыми типами:

// ОШИБКА! Компилятор не поймёт, какой метод вызывать

```
public static int getValue() { return 5; }  
public static double getValue() { return 5.0; }
```

5) Логическая декомпозиция: разбиение задачи на методы

Теперь, когда мы знаем, как создавать методы, научимся правильно разбивать программу на маленькие, понятные части.

Плохой пример (всё в main)

java

Копировать

Скачать

```
public class BadExample {  
    public static void main(String[] args) {  
        // Ввод данных  
        Scanner scanner = new Scanner(System.in);  
        System.out.print("Введите первое число: ");  
        int a = scanner.nextInt();  
        System.out.print("Введите второе число: ");  
        int b = scanner.nextInt();  
        System.out.print("Введите операцию (+, -, *, /): ");  
        char op = scanner.next().charAt(0);  
  
        // Вычисление  
        double result = 0;  
        switch (op) {  
            case '+': result = a + b; break;  
            case '-': result = a - b; break;  
            case '*': result = a * b; break;  
            case '/':  
                if (b != 0) result = (double) a / b;  
                else System.out.println("Ошибка!");  
                break;  
            default: System.out.println("Неизвестная операция");  
        }  
  
        // Вывод  
        System.out.println("Результат: " + result);  
    }  
}
```

Хороший пример (разбиение на методы)

```

import java.util.Scanner;

public class GoodExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int a = getNumber(scanner, "первое");
        int b = getNumber(scanner, "второе");
        char operation = getOperation(scanner);

        double result = calculate(a, b, operation);

        if (Double.isNaN(result)) {
            System.out.println("Ошибка при вычислении");
        } else {
            displayResult(a, b, operation, result);
        }

        scanner.close();
    }

    // Метод для ввода числа
    public static int getNumber(Scanner scanner, String order) {
        System.out.print("Введите " + order + " число: ");
        return scanner.nextInt();
    }

    // Метод для ввода операции
    public static char getOperation(Scanner scanner) {
        System.out.print("Введите операцию (+, -, *, /): ");
        return scanner.next().charAt(0);
    }

    // Метод для вычисления
    public static double calculate(int a, int b, char operation) {
        switch (operation) {
            case '+': return a + b;
            case '-': return a - b;
            case '*': return a * b;
            case '/':
                if (b != 0) return (double) a / b;
                else return Double.NaN; // Not a Number — сигнал об ошибке
            default: return Double.NaN;
        }
    }
}

```



```

// Метод для вывода результата
public static void displayResult(int a, int b, char op, double result) {
    System.out.println(a + " " + op + " " + b + " = " + result);
}
}

```

Принципы хорошей декомпозиции:

1. **Один метод — одна задача** (Single Responsibility Principle)
2. **Имя метода должно отражать его действие** (глагол)
3. **Метод должен быть достаточно коротким** (10-20 строк — хорошо)
4. **Избегайте дублирования кода** — выносите повторяющееся в метод

б) Рекурсия (метод вызывает сам себя)

Рекурсия — это техника, при которой метод вызывает сам себя. Рекурсия позволяет элегантно решать задачи, которые можно разбить на подзадачи того же типа.

Основные правила рекурсии:

1. **Базовый случай** — условие, при котором рекурсия прекращается
2. **Рекурсивный случай** — метод вызывает сам себя с изменёнными параметрами
3. **Каждый вызов должен приближать к базовому случаю** (иначе бесконечная рекурсия)

Пример 1: Факториал ($n! = n \times (n-1)!$)

```

public class FactorialRecursion {
    public static void main(String[] args) {
        System.out.println("5! = " + factorial(5)); // 120
        System.out.println("0! = " + factorial(0)); // 1
        System.out.println("7! = " + factorial(7)); // 5040
    }

    public static int factorial(int n) {
        // Базовый случай
        if (n == 0 || n == 1) {
            return 1;
        }
        // Рекурсивный случай: n! = n * (n-1)!
        return n * factorial(n - 1);
    }
}

```

```
}  
}
```

Как работает рекурсия для factorial(3):

text

Копировать

Скачать

factorial(3)

→ 3 * factorial(2)

→ 3 * (2 * factorial(1))

→ 3 * (2 * 1)

→ 3 * 2

→ 6

Пример 2: Числа Фибоначчи ($F(n) = F(n-1) + F(n-2)$)

```
public class FibonacciRecursion {  
    public static void main(String[] args) {  
        // Вывод первых 10 чисел Фибоначчи  
        for (int i = 0; i < 10; i++) {  
            System.out.print(fibonacci(i) + " ");  
        }  
        // 0 1 1 2 3 5 8 13 21 34  
    }  
  
    public static int fibonacci(int n) {  
        // Базовые случаи  
        if (n == 0) return 0;  
        if (n == 1) return 1;  
        // Рекурсивный случай  
        return fibonacci(n - 1) + fibonacci(n - 2);  
    }  
}
```

Пример 3: Сумма цифр числа

```
public class DigitSumRecursion {  
    public static void main(String[] args) {  
        System.out.println("Сумма цифр 1234: " + sumDigits(1234)); // 10  
        System.out.println("Сумма цифр 0: " + sumDigits(0)); // 0  
        System.out.println("Сумма цифр 999: " + sumDigits(999)); // 27  
    }  
  
    public static int sumDigits(int n) {
```

```

// Базовый случай: если число меньше 10, возвращаем его
if (n < 10) {
    return n;
}
// Рекурсивный случай: последняя цифра + сумма остальных
return (n % 10) + sumDigits(n / 10);
}
}

```

Рекурсия имеет свои преимущества и недостатки:

Преимущества	Недостатки
Код становится короче и понятнее для некоторых задач	Может быть медленнее из-за накладных расходов на вызовы
Естественно описывает рекурсивные структуры (деревья, графы)	Большое количество вызовов может привести к переполнению стека (StackOverflowError)
Упрощает доказательство корректности	Требует больше памяти для хранения стека вызовов

Таблица 2. Преимущества и недостатки рекурсии в Java

Когда использовать рекурсию:

- Задача естественно разбивается на подзадачи того же типа (обход деревьев, вычисление факториала, ханойские башни)
- Глубина рекурсии не слишком велика (обычно < 1000-5000)

Когда использовать итерации (циклы):

- Простые повторяющиеся вычисления
- Большая глубина рекурсии
- Критична производительность

7) Типичные ошибки при работе с методами

Ошибка 1: Забыли static при вызове из main

```

// ОШИБКА
public class Test {
    public static void main(String[] args) {
        sayHello(); // Нестатический метод нельзя вызвать из статического
    }
}

```

```

    }

    public void sayHello() {
        System.out.println("Hello");
    }
}

// ПРАВИЛЬНО (два варианта)
public class Test {
    public static void main(String[] args) {
        sayHello();           // вариант 1: сделать метод статическим
        new Test().sayHello(); // вариант 2: создать объект и вызвать
    }

    public static void sayHello() { ... } // статический метод
    // или
    public void sayHello() { ... }      // нестатический
}

```

Ошибка 2: Несовпадение типов при вызове

```

java
Копировать
Скачать
public static int add(int a, int b) {
    return a + b;
}

public static void main(String[] args) {
    double result = add(5, 3); // ОК (int -> double)
    int result2 = add(5.5, 3); // ОШИБКА! double не может стать int автома
тически
    int result3 = add(5, 3);   // ОК
}

```

Ошибка 3: Не все пути возвращают значение

```

// ОШИБКА
public static int absolute(int x) {
    if (x >= 0) {
        return x;
    }
    // Если x < 0, метод ничего не возвращает!
}

```

// ПРАВИЛЬНО

```
public static int absolute(int x) {  
    if (x >= 0) {  
        return x;  
    } else {  
        return -x;  
    }  
}
```

Ошибка 4: Бесконечная рекурсия

// ОШИБКА — нет базового случая

```
public static int infiniteRecursion(int n) {  
    return n + infiniteRecursion(n - 1); // Никогда не остановится  
}
```

// ПРАВИЛЬНО

```
public static int sumToN(int n) {  
    if (n <= 0) return 0; // Базовый случай  
    return n + sumToN(n - 1); // Рекурсивный случай  
}
```

Итоги восьмой лекции

Сегодня мы изучили:

Методы (подпрограммы):

- Разбивают программу на логические блоки
- Позволяют повторно использовать код
- Упрощают чтение, тестирование и отладку

Компоненты метода:

- Модификаторы (public, static)
- Возвращаемый тип (void, int, double, String, boolean и др.)
- Имя метода (глагол, camelCase)
- Параметры (входные данные)
- Тело метода (код)
- return (возврат значения или выход из метода)

Перегрузка методов:

- Несколько методов с одинаковым именем

- Разные параметры (количество, тип, порядок)
- Компилятор выбирает нужный по аргументам

Рекурсия:

- Метод вызывает сам себя
- Должен быть базовый случай (остановка)
- Каждый вызов приближает к базовому случаю

Ключевые выводы:

1. Методы — основа структурного программирования
2. Один метод должен решать одну задачу
3. Имена методов должны отражать их действие
4. Перегрузка делает интерфейс методов удобным
5. Рекурсия элегантна, но требует осторожности (риск переполнения стека)

Связь с итоговым проектом

Теперь вы можете улучшить «Аграрный калькулятор», разбив его на методы:

```
public class AgrarianCalculator {
    public static void main(String[] args) {
        // Только управление меню
    }

    public static void showMenu() { ... }
    public static double calculateSeeds(double area, double normPerHa) { ... }
    public static double calculateFertilizers(String crop, double area) { ... }
    public static double calculateYield(double area, double yieldPerHa, String weather) { ... }
    public static void analyzeFields() { ... }
    public static void saveResult(String data) { ... }
    public static void loadResults() { ... }
    public static double getPositiveDouble(Scanner sc, String prompt) { ... }
    public static int getMenuChoice(Scanner sc, int min, int max) { ... }
}
```

Такой подход делает код:

- **Читаемым** — каждая часть понятна по названию метода
- **Тестируемым** — можно проверить каждый метод отдельно
- **Поддерживаемым** — легко найти и исправить ошибку
- **Расширяемым** — легко добавить новые функции